

“The Essentials of Computer Organization and Architecture”

Unidade 4 – Exercícios & Soluções

LEBE1 Num computador os inteiros são de 32 bits e a memória é endereçável ao byte. Mostre como o valor 1234_{16} é guardado na memória desse computador, a partir do endereço 0_2 , se:

- a) O computador seguir a convenção Big Endian
- b) O computador seguir a convenção Little Endian

LEBE1 Num computador os inteiros são de 32 bits e a memória é endereçável ao byte. Mostre como o valor 1234_{16} é guardado na memória desse computador, a partir do endereço 0_2 , se:

- a) O computador seguir a convenção Big Endian
- b) O computador seguir a convenção Little Endian

Solução:

Address →	00	01	10	11
Big Endian	00	00	12	34
Little Endian	34	12	00	00

LEBE2 Num computador os inteiros são de 32 bits e a memória é endereçável ao byte. Mostre como cada valor hexadecimal fornecido abaixo é guardado na memória desse computador, a partir do endereço 10_{16} , para as alternativas little endian e big endian.

a) **$456789A1_{16}$**

b) **$0000058A_{16}$**

c) **14148888_{16}**

LEBE2 Num computador os inteiros são de 32 bits e a memória é endereçável ao byte. Mostre como cada valor hexadecimal fornecido abaixo é guardado na memória desse computador, a partir do endereço 10_{16} , para as alternativas little endian e big endian.

a) $456789A1_{16}$

b) $0000058A_{16}$

c) 14148888_{16}

Solução: a)

Address →	10_{16}	11_{16}	12_{16}	13_{16}
Big Endian	45	67	89	A1
Little Endian	A1	89	67	45

b)

Address →	10_{16}	11_{16}	12_{16}	13_{16}
Big Endian	00	00	05	8A
Little Endian	8A	05	00	00

c)

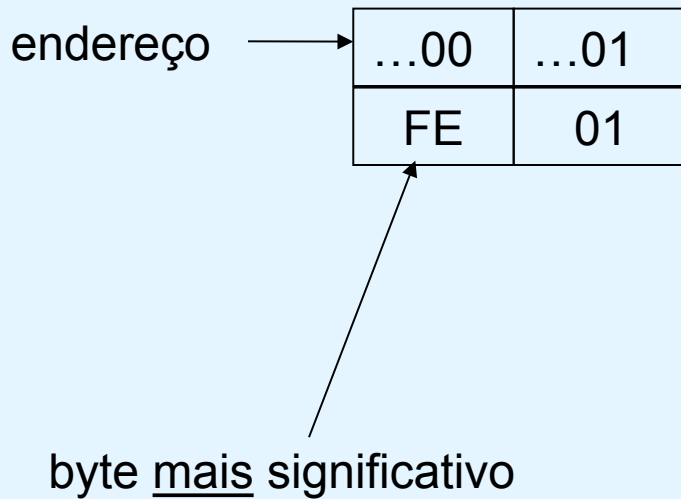
Address →	10_{16}	11_{16}	12_{16}	13_{16}
Big Endian	14	14	88	88
Little Endian	88	88	14	14

LEBE3 Os primeiros dois bytes de uma memória de 2Mx16bit guardam os valores FE_{16} e 01_{16} . Juntos, esses dois bytes representam um inteiro em complemento para 2. Qual o valor em decimal desse inteiro:

- a) Se esse inteiro foi guardado em formato Big Endian
- b) Se esse inteiro foi guardado em formato Little Endian

LEBE3 Os primeiros dois bytes de uma memória de 2Mx16bit guardam os valores FE_{16} e 01_{16} . Juntos, esses dois bytes representam um inteiro em complemento para 2. Qual o valor em decimal desse inteiro:

a) Big Endian – Solução:



o valor é **negativo**

$$FE01_{16} = 1111\ 1110\ 0000\ 0001_2$$

descoberta do valor absoluto do inteiro:

complemento para 1:

$$1111\ 1110\ 0000\ 0001_2 \rightarrow$$

$$0000\ 0001\ 1111\ 1110_2$$

soma de uma unidade:

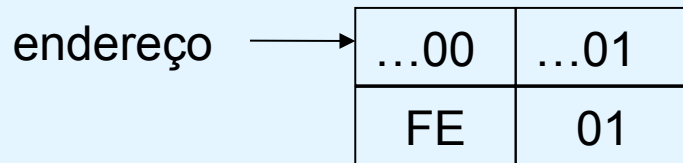
$$0000\ 0001\ 1111\ 1110_2 + 1_2 =$$

$$0000\ 0001\ 1111\ 1111_2 = 511_{10}$$

conclusão: o valor guardado é o -511_{10}

LEBE3 Os primeiros dois bytes de uma memória de 2Mx16bit guardam os valores FE_{16} e 01_{16} . Juntos, esses dois bytes representam um inteiro em complemento para 2. Qual o valor em decimal desse inteiro:

b) Little Endian – Solução:



o valor é **positivo**

$$01FE_{16} = \textcolor{red}{0}000\ 0001\ 1111\ 1110_2 = 510_{10}$$

conclusão: o valor guardado é o 510_{10}

LEBE4 Num computador de arquitetura Little-Endian e números inteiros de 16 bits (representados em sinal e magnitude), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 1234AABB.

- a) Quantos inteiros de 16 bits estão guardados nos primeiros 4 bytes da memória RAM ?
- b) Extraia da memória e reconstrua os números inteiros identificados em a). Apresente esses números, em hexadecimal, da forma como é comum os humanos representarem os números (ou seja, em Big-Endian).
- c) Apresente, em decimal, os números identificados em b).
- d) Some os dois números apresentados em c), e apresente agora o valor da sua soma, em hexadecimal, da forma como é comum os humanos representarem os números (ou seja, em Big-Endian). Nota: no caso da soma produzir overflow/underflow, preserve o sinal (ou seja, não use o bit de sinal para acomodar o overflow/underflow).
- e) Mostre como seria armazenado na memória RAM deste computador o valor da soma apresentado em d), assumindo que o armazenamento se faz a partir do byte 100.

LEBE4 Num computador de arquitetura Little-Endian e números inteiros de 16 bits (representados em sinal e magnitude), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 1234AABB.

a) Quantos inteiros de 16 bits estão guardados nos primeiros 4 bytes da memória RAM ?

Solução:

2

Justificação: se os números inteiros são de 16 bits, isso corresponde a 2 bytes; donde, nos primeiros 4 bytes da memória estarão 2 números (um número por cada 2 bytes).

LEBE4 Num computador de arquitetura Little-Endian e números inteiros de 16 bits (representados em sinal e magnitude), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 1234AABB.

b) Extraia da memória e reconstrua os números inteiros identificados em a). Apresente esses números, em hexadecimal, da forma como é comum os humanos representarem os números (ou seja, em Big-Endian).

Solução:

3412

BBAA

Justificação: na memória estão então os números 1234 (16) e AABB (16) ; como estão guardados em Little-Endian, originalmente esses números eram 3412 (16) e BBAA (16); notar que cada número ocupa 2 bytes, e cada byte necessita de 2 dígitos hexadecimais.

LEBE4 Num computador de arquitetura Little-Endian e números inteiros de 16 bits (representados em sinal e magnitude), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 1234AABB.

c) Apresente, em decimal, os números identificados em b).

Solução:

13330

-15274

Justificação: já vimos que o primeiro número na memória é 3412 (16) e o segundo é BBAA (16); considerando que estão em sinal e magnitude, o valor na base 10 é:

- $3412_{(16)} = \mathbf{0}011\ 0100\ 0001\ 0010_{(2)} = \mathbf{+}\ 011\ 0100\ 0001\ 0010_{(2)} = 13330_{(10)}$
- $BBAA_{(16)} = \mathbf{1}011\ 1011\ 1010\ 1010_{(2)} = \mathbf{-}\ 011\ 1011\ 1010\ 1010_{(2)} = -15274_{(10)}$

LEBE4 Num computador de arquitetura Little-Endian e números inteiros de 16 bits (representados em sinal e magnitude), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 1234AABB.

d) Some os dois números apresentados em c), e apresente agora o valor da sua soma, em hexadecimal, da forma como é comum os humanos representarem os números (ou seja, em Big-Endian). Nota: no caso da soma produzir overflow/underflow, preserve o sinal (ou seja, não use o bit de sinal para acomodar o overflow/underflow).

Solução:

8798

Justificação: os números identificados na alínea c) são 13330 (10) e -15274 (10) e a sua soma dá -1944 (10); em binário (e usando apenas 15 bits, pois o 16º será para o **sinal**), o módulo deste número corresponde a 000 0111 1001 1000 (2); acrescentando o **bit de sinal**, que é **1** (pois o número é negativo), o valor da soma em sinal e magnitude com 16 bits fica -1944 (10) = **1**000 0111 1001 1000 (2) = **8798** (16).

LEBE4 Num computador de arquitetura Little-Endian e números inteiros de 16 bits (representados em sinal e magnitude), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 1234AABB.

e) Mostre como seria armazenado na memória RAM deste computador o valor apresentado em d), assumindo que o armazenamento se faz a partir do byte 100.

Solução:

Byte 100: 98

Byte 101: 87

Justificação: o número calculado em d) é 8798 (16) e esta representação corresponde à que os humanos costumam usar (Big Endian); como o número tem de ser guardado na memória em Little Endian, primeiro guarda-se 98 (parte menos significativa) e só depois se guarda 87 (parte mais significativa).

LEBE5 Num computador de arquitetura Big-Endian e números inteiros de 16 bits (representados em complemento $p/2$), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 98000089.

- a)** Quantos inteiros de 16 bits estão guardados nos primeiros 4 bytes da memória RAM ?
- b)** Extraia da memória e reconstrua os números inteiros identificados em a). Apresente esses números, em hexadecimal, da forma como é comum os humanos representarem os números (ou seja, em Big-Endian).
- c)** Apresente, em decimal, os números identificados em b).
- d)** Some os dois números apresentados em c), e apresente agora o valor da sua soma, em hexadecimal, da forma como é comum os humanos representarem os números (ou seja, em Big-Endian). Nota: no caso da soma produzir overflow/underflow, preserve o sinal (ou seja, não use o bit de sinal para acomodar o overflow/underflow).
- e)** Mostre como seria armazenado na memória RAM deste computador o valor da soma apresentado em d), assumindo que o armazenamento se faz a partir do byte 10F.

LEBE5 Num computador de arquitetura Big-Endian e números inteiros de 16 bits (representados em complemento p/ 2), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 98000089.

a) Quantos inteiros de 16 bits estão guardados nos primeiros 4 bytes da memória RAM ?

Solução:

2

Justificação: se os números inteiros são de 16 bits, isso corresponde a 2 bytes; donde, nos primeiros 4 bytes da memória estarão 2 números (um número por cada 2 bytes).

LEBE5 Num computador de arquitetura Big-Endian e números inteiros de 16 bits (representados em complemento p/ 2), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 98000089.

b) Extraia da memória e reconstrua os números inteiros identificados em a). Apresente esses números, em hexadecimal, da forma como é comum os humanos representarem os números (ou seja, em Big-Endian).

Solução:

9800

0089

Justificação: na memória estão então os números 9800 (16) e 0089 (16) ; como estão guardados em Big Endian, originalmente esses números eram 9800 (16) e 0089 (16); notar que cada número ocupa 2 bytes, e cada byte necessita de 2 dígitos hexadecimais.

LEBE5 Num computador de arquitetura Big-Endian e números inteiros de 16 bits (representados em complemento p/ 2), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 98000089.

c) Apresente, em decimal, os números identificados em b).

Solução:

-26624

137

Justificação: já vimos que o primeiro número na memória é 9800 (16) e o segundo é 0089 (16); considerando que estão em complemento para 2, o valor na base 10 é:

- $9800 (16) = 1001\ 1000\ 0000\ 0000 (2)$; sendo um número negativo em complemento para 2, o positivo correspondente obtém-se invertendo os bits e somando 1, ou seja, $0110\ 0111\ 1111\ 1111 + 1 = 0110\ 1000\ 0000\ 0000 (2) = + 26624 (10)$, donde $9800 (16) = - 26624 (10)$
- $0089 (16) = 0000\ 0000\ 1000\ 1001 (2)$; sendo um número positivo em complemento para 2, o seu valor é de leitura direta, ou seja, $0089 (16) = 137 (10)$

LEBE5 Num computador de arquitetura Big-Endian e números inteiros de 16 bits (representados em complemento p/ 2), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 98000089.

d) Some os dois números apresentados em c), e apresente agora o valor da sua soma, em hexadecimal, da forma como é comum os humanos representarem os números (ou seja, em Big-Endian). Nota: no caso da soma produzir overflow/underflow, preserve o sinal (ou seja, não use o bit de sinal para acomodar o overflow/underflow).

Solução:

9889

Justificação: os números identificados na alínea c) são -26624 (10) e 137 (10) e a sua soma dá -26487 (10); sendo este um número negativo, para saber como se representa em complemento para 2, representa-se o seu módulo em binário (usando só 15 bits, pois o 16º será para o sinal), invertem-se os seus bits e soma-se 1: $26487(10) = 110\ 0111\ 0111\ 0111\ (2)$; invertendo tem-se $001\ 1000\ 1000\ 1000\ (2)$ e somando 1 tem-se $001\ 1000\ 1000\ 1001\ (2)$; acrescentando o bit de sinal, que é 1 (pois o número é negativo), obtemos $-26487\ (10) = 1001\ 1000\ 1000\ 1001\ (2) = \mathbf{9889}\ (16)$.

LEBE5 Num computador de arquitetura Big-Endian e números inteiros de 16 bits (representados em complemento $p/2$), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 98000089.

e) Mostre como seria armazenado na memória RAM deste computador o valor da soma apresentado em d), assumindo que o armazenamento se faz a partir do byte 10F.

Solução:

Byte 10F: 98

Byte 110: 89

Justificação: o número calculado em d) é 9889 (16) e esta representação corresponde à que os humanos costuma usar (Big Endian); como o número tem de ser guardado na memória também em Big Endian, primeiro guarda-se 98 (parte mais significativa) e só depois se guarda 89 (parte menos significativa).

LEBE6 Que problemas poderão surgir no contexto de uma transferência de dados através da rede, entre máquinas com endianness diferente, se não houver precauções? Exemplifique com os dados de LEBE3.

LEBE6 Que problemas poderão surgir no contexto de uma transferência de dados através da rede, entre máquinas com endianness diferente, se não houver precauções? Exemplifique com os dados de LEBE3.

Problema:

Os valores serão erradamente interpretados, se não forem sujeitos às conversões necessárias antes do envio / após a receção. Exemplo com base nos valores do exercício **LEBE3**:

Sender	Receiver
Big	Little
-511	510
510	-511

Sender	Receiver
Little	Big
510	-511
-511	510

Solução:

- assume-se que a rede é Big-Endian (BE), e os hosts BE ou Little-Endian (LE)
- uma máquina BE não necessita de converter os dados antes do envio (cópia direta da memória p/ rede), nem após a receção (cópia direta da rede p/ memória)
- uma máquina LE tem de converter os dados para BE antes do envio (antes da cópia p/ rede), e de BE para LE após a receção (antes de guardar na memória)

LEBE7 Escreva, em linguagem C, um programa que permita saber se a arquitetura da máquina executante segue o formato Big Endian ou o formato Little Endian.

LEBE7 Escreva, em linguagem C, um programa que permita saber se a arquitetura da máquina executante segue o formato Big Endian ou o formato Little Endian.

Solução:

```
#include <stdio.h>
int main()
{
    int a = 0x80000000; //32bit
    char *b = (char *) &a;
    if(b[0] != 0)
        printf("Big-Endian\n");
    else
        printf("Little-Endian\n");
}
```

SBA1 Converta as seguintes expressões em notação infix, para notação postfix.

a. $X * Y + W * Z + V * U$

b. $W * X + W * (U * V + Z)$

c. $(W * (X + Y * (U * V))) / (U * (X + Y))$

Nota:

- utilize o algoritmo do próximo slide, apropriado a uma implementação computacional (TPC: aplicar o algoritmo pelo menos à expressão a.) ...
- ... ou, em alternativa, aplique a heurística ensinada nas aulas.

Algoritmo de Conversão INFIX para POSTFIX

Notação: Q = expressão *infix*; P = expressão *postfix* equivalente a Q

1. acrescentar “)” a Q e inserir (PUSH) “(“ numa stack vazia
2. **Para** cada símbolo de Q (lido da esquerda para a direita), **Repetir** os passos 3 a 6, **Até** que a stack fique vazia
3. **Se** o próximo símbolo é “(“, **Então** inseri-lo (PUSH) na stack
4. **Se** o próximo símbolo é um operando, **Então** adicioná-lo a P
5. **Se** o próximo símbolo é um operador, **Então**
 - a. retirar (POP) da stack e adicionar a P cada operador do topo da stack com igual ou maior precedência que a do operador encontrado em Q
 - b. inserir (PUSH) na stack o operador encontrado em Q
6. **Se** o próximo símbolo é “)“, **Então**
 - a. retirar (POP) da stack e adicionar a P cada operador do topo da stack, até se encontrar um símbolo “(“ no topo da stack
 - b. retirar (POP) o símbolo “(“ do topo da stack, não o adicionando a P

SBA1 Converta as seguintes expressões em notação infix, para postfix.

a. $X * Y + W * Z + V * U$

b. $W * X + W * (U * V + Z)$

c. $(W * (X + Y * (U * V))) / (U * (X + Y))$

Soluções:

a. $X Y * W Z * + V U * +$

b. $W X * W U V * Z + * +$

c. $W X Y U V * * + * U X Y + * /$

SBA2 Converta as seguintes expressões em notação postfix, para infix.

a. W X Y Z - + *

b. U V W X Y Z + * + * +

c. X Y Z + V W - * Z + +

Nota:

- utilize o algoritmo do próximo slide, apropriado a uma implementação computacional; basicamente, é o algoritmo usado para avaliar uma expressão infix numa arquitetura baseada em stack; no final do processo, a expressão guardada na stack corresponde à versão postfix da expressão infix original

Algoritmo de Conversão POSTFIX para INFIX

Notação: P = expressão *postfix* ; Q = expressão *infix* equivalente a P

1. criar uma stack vazia
2. **Para** cada símbolo de P (lido da esquerda para a direita), **Repetir** os passos 3 a 4, **Até** que a stack fique com uma só expressão (que será a expressão Q)
3. **Se** o próximo símbolo é um operando, **Então** inseri-lo (PUSH) na stack
4. **Se** o próximo símbolo é um operador, **Então**
 - a. retirar (POP) duas expressões do topo da stack
 - b. aplicar o operador às duas expressões anteriores
 - c. inserir (PUSH) na stack a expressão resultante

SBA2 Converta as seguintes expressões em notação postfix, para infix.

a. W X Y Z - + *

b. U V W X Y Z + * + * +

c. X Y Z + V W - * Z + +

Soluções:

a. $W * (X + (Y - Z))$

b. $U + V * (W + X * (Y + Z))$

c. $X + ((Y + Z) * (V - W) + Z)$

SBA3 a) converta a seguinte expressão de infix para postfix.

$$X = \frac{A - B + C * (D * E - F)}{G + H * K}$$

SBA3 a) converta a seguinte expressão de infix para postfix.

$$X = \frac{A - B + C * (D * E - F)}{G + H * K}$$

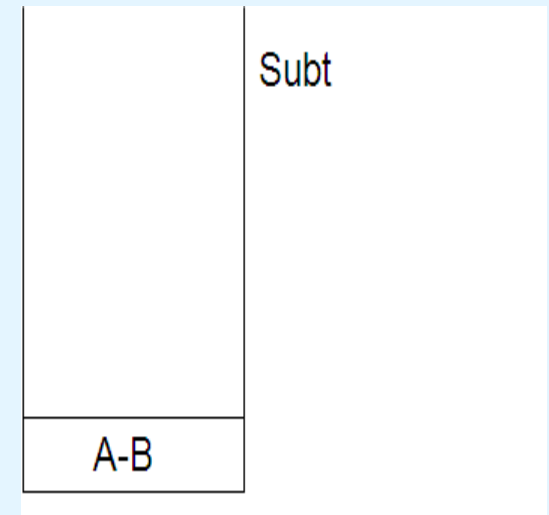
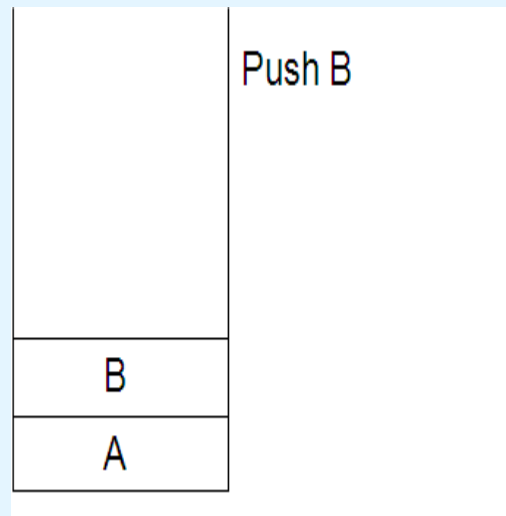
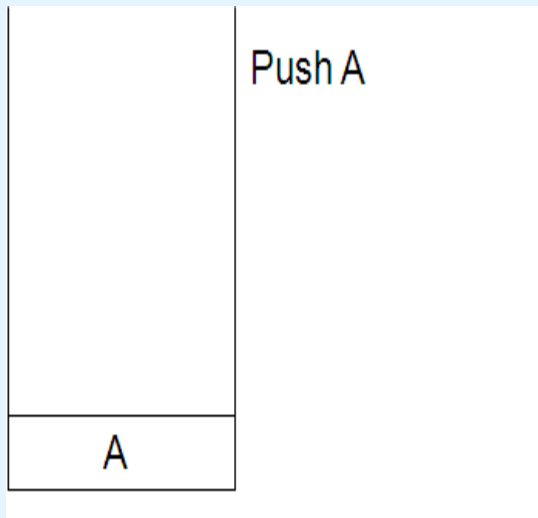
Soluções – a)

A B - C D E * F - * + G H K * + /

SBA3 b) escreva um programa assembly que calcule o valor de X usando uma arquitetura baseada em stack.

Soluções – b)

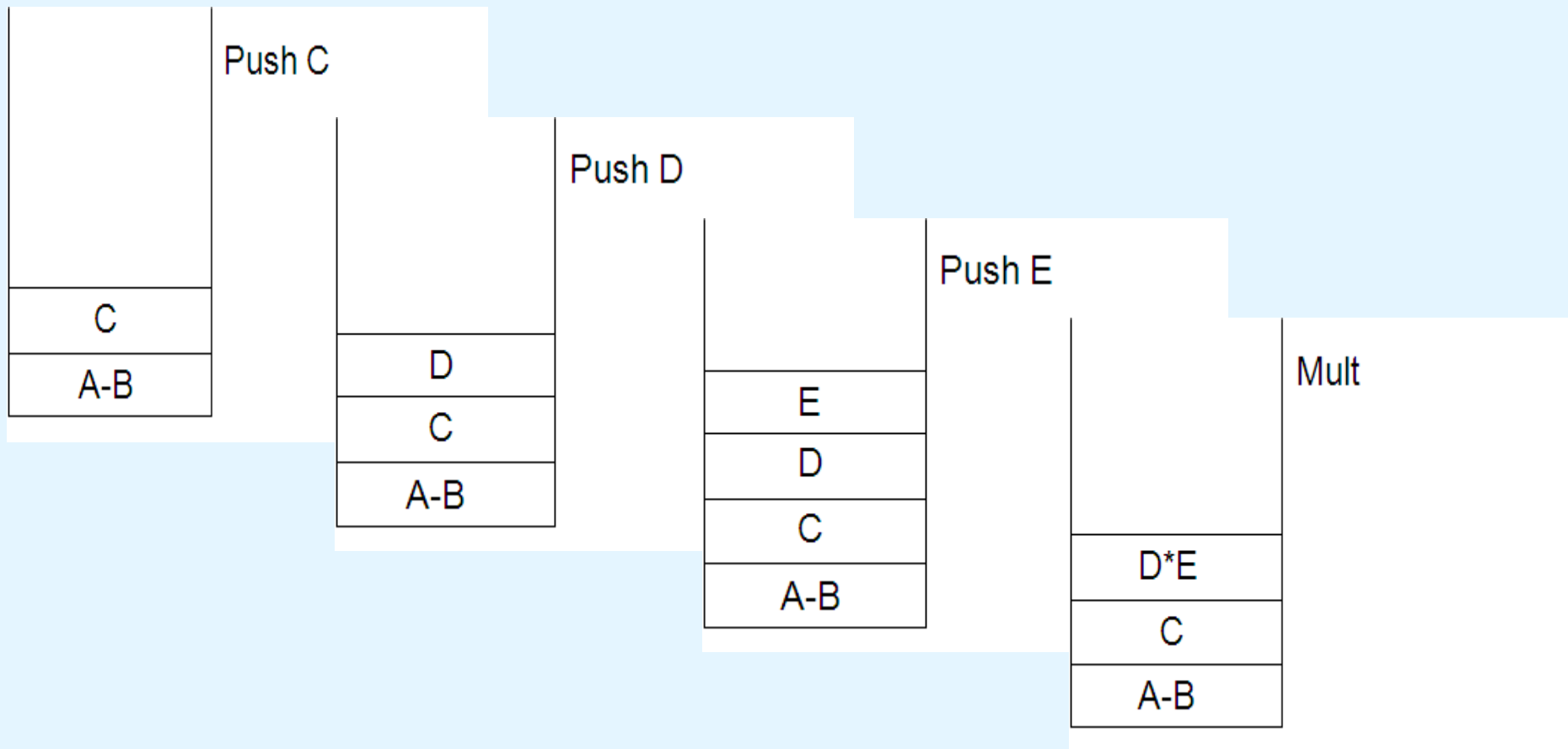
A **B** - C D E * F - * + G H K * + /



SBA3 b) escreva um programa assembly que calcule o valor de X usando uma arquitetura baseada em stack.

Soluções – b)

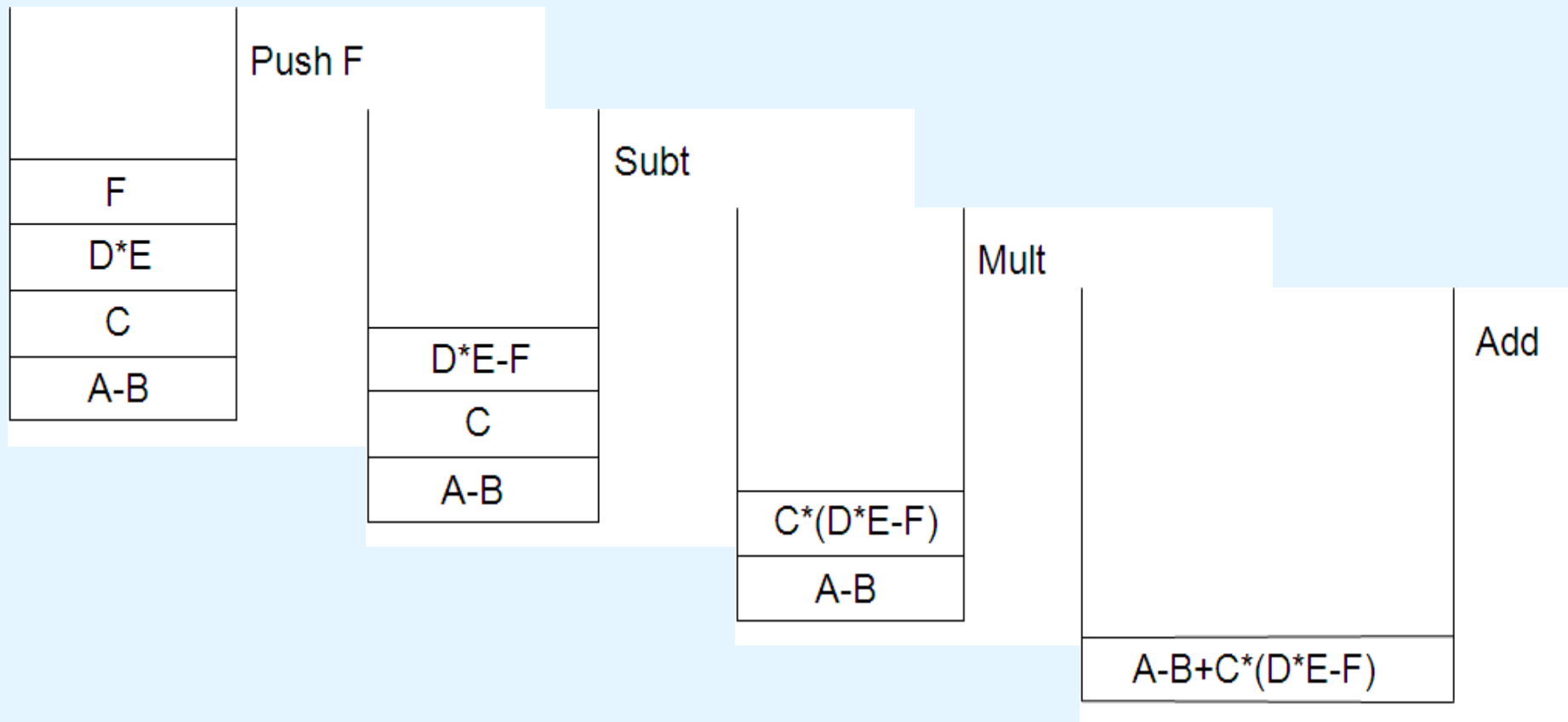
A B – C D E * F - * + G H K * + /



SBA3 b) escreva um programa assembly que calcule o valor de X usando uma arquitetura baseada em stack.

Soluções – b)

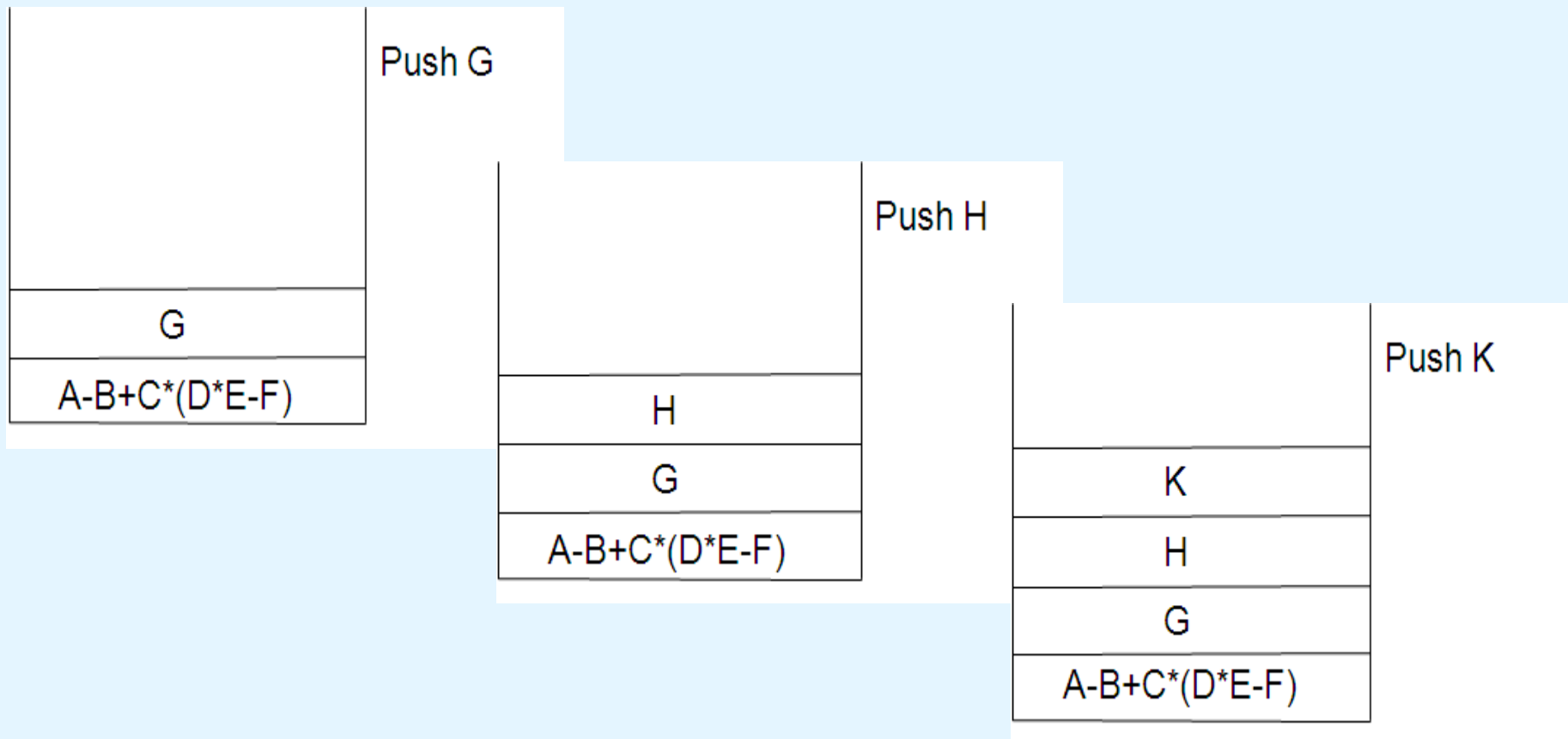
A B – C D E * F - * + G H K * + /



SBA3 b) escreva um programa assembly que calcule o valor de X usando uma arquitetura baseada em stack.

Soluções – b)

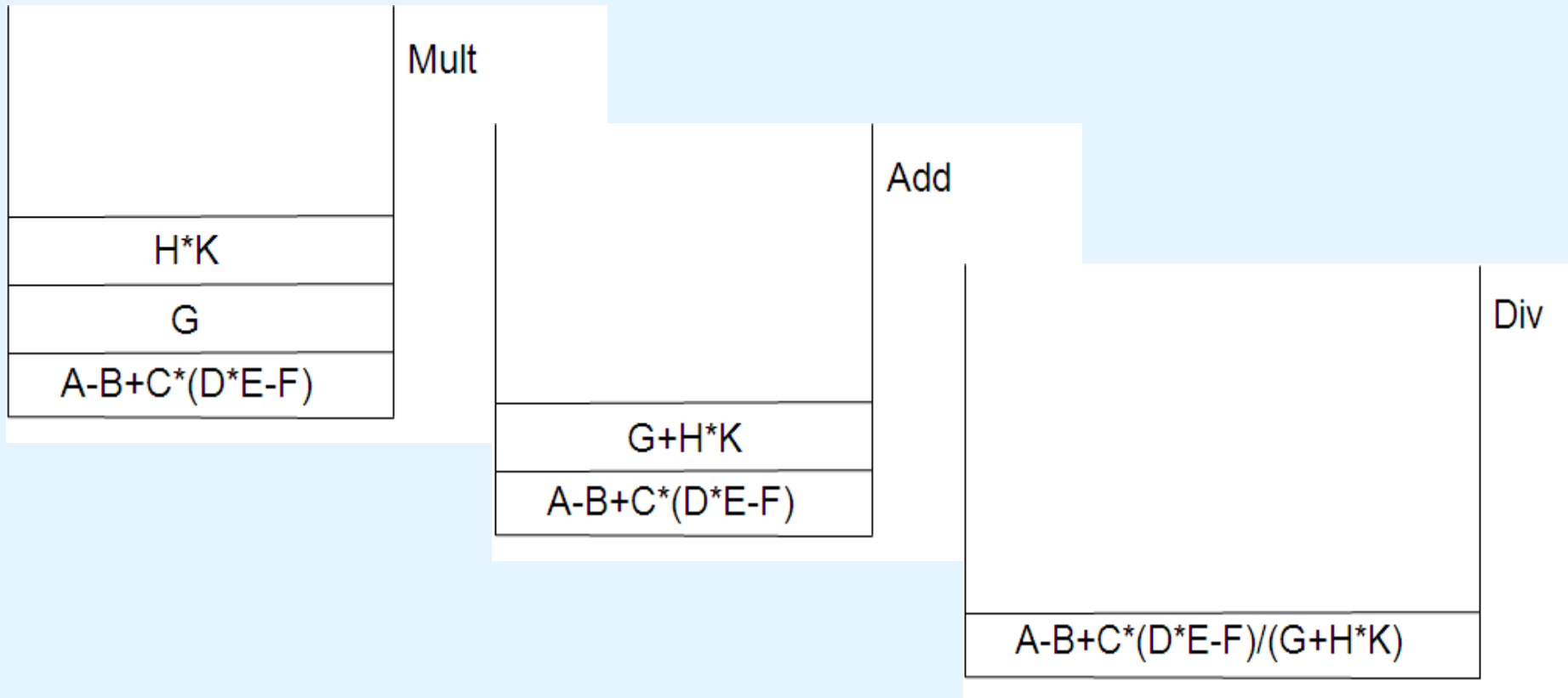
A B - C D E * F - * + G H K * + /



SBA3 b) escreva um programa assembly que calcule o valor de X usando uma arquitetura baseada em stack.

Soluções – b)

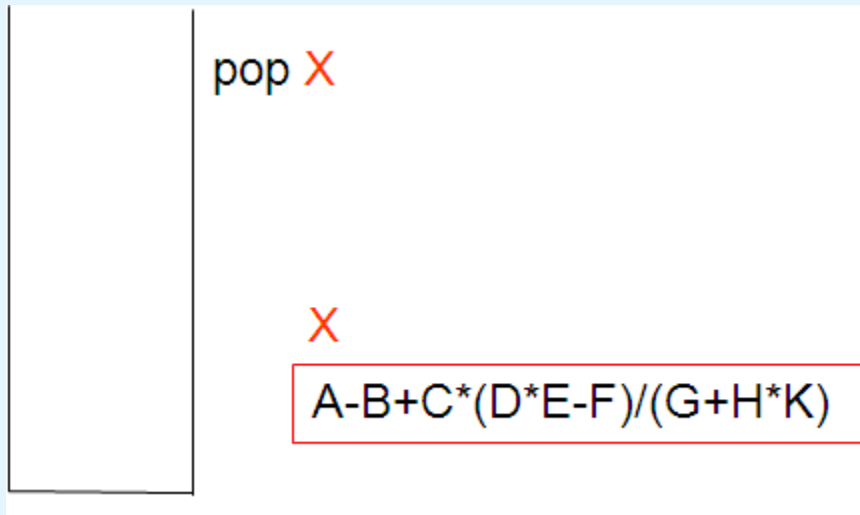
A B - C D E * F - * + G H K * + /



SBA3 b) escreva um programa assembly que calcule o valor de X usando uma arquitetura baseada em stack.

Soluções – b)

$$X = A B - C D E * F - * + G H K * + /$$



```
Push A
Push B
Subtract
Push C
Push D
Push E
Mult
Push F
Subtract
Mult
Add
Push G
Push H
Push K
Mult
Add
Div
Pop X
```

NOIS1 Escreva código assembly que permita avaliar a expressão $A = (B+C)*(D+E)$ em máquinas de 0, 1, 2 ou 3 endereços. Nota: preserve os valores de B, C, D e E.

NOIS1 Escreva código assembly que permita avaliar a expressão $A = (B+C)*(D+E)$ em máquinas de 0, 1, 2 ou 3 endereços. Nota: preserve os valores de B, C, D e E.

Soluções:

Add R1, B, C
Add R2, D, E
Mult A, R1, R2

3-address machine

Load R1, B
Add R1, C
Load R2, D
Add R2, E
Mult R2, R1
Store A, R2

2-address machine

Load B
Add C
Store Temp
Load D
Add E
Mult Temp
Store A

1-address machine

Push B
Push C
Add
Push D
Push E
Add
Mult
Pop A

0-address machine

NOIS2 Recorde o exercício NOIS1 e apresente uma estimativa do espaço mínimo necessário (em bytes) para suportar o programa correspondente a cada um dos quatro cenários, assumindo que a quantidade máxima de RAM endereçável é de 16K palavras.

NOIS2 Recorde o exercício NOIS1 e apresente uma estimativa do espaço mínimo necessário (em bytes) para suportar o programa correspondente a cada um dos quatro cenários, assumindo que a quantidade máxima de RAM endereçável é de 16K palavras.

SOLUÇÃO:

pressuposto comum a todos os cenários:

- para cada cenário, o número de bits das instruções será fixo (i.e., todas as instruções usadas nesse cenário têm o mesmo número de bits, mesmo que isso implique “desperdício” de bits em algumas instruções)

facto comum a todos os cenários:

- não existindo mais de 16K palavras de memória, um endereço dessa memória não necessitará de mais de 14 bits (assumindo endereçamento à palavra):
 $\#PALAVRAS(RAM)=16K$
 $\#BITS(end. RAM)=\log_2(16K) = \log_2(2^4 \cdot 2^{10})=14 \text{ bits}$
- portanto, sempre que for necessário, numa instrução, referenciar um endereço de memória, haverá que utilizar 14 bits dessa instrução para o efeito

NOIS2 Recorde o exercício NOIS1 e apresente uma estimativa do espaço mínimo necessário (em bytes) para suportar o programa correspondente a cada um dos quatro cenários, assumindo que a quantidade máxima de RAM endereçável é de 16K palavras.

SOLUÇÃO:

a) cenário “0-address-machine”

- formatos de instrução: |opcode|endereço| (*) , |opcode| (**)
- assumindo que a ISA tem apenas 4 instruções (ADD, MULT, PUSH e POP), são apenas necessários 2 bits para o opcode:
#INST(ISA) = 4 instruções
 $\#BITS(opcode) = \log_2(\#INST(ISA)) = \log_2(4) = 2 \text{ bits}$
- as instruções que necessitam de mais bits são as instruções (*):
 - 2 bits para o opcode
 - 14 bits para 1 end. de memória (A a E)num total de 16 bits = 2 bytes
- o número de bits por instrução é fixo => todas as instruções terão 2 bytes
- contendo **8 instruções**, o programa ocupará $8 * 2 = \mathbf{16 \text{ bytes}}$

Push B	(*)
Push C	(*)
Add	(**)
Push D	(*)
Push E	(*)
Add	(**)
Mult	(**)
Pop A	(*)

0-address machine

NOIS2 Recorde o exercício NOIS1 e apresente uma estimativa do espaço mínimo necessário (em bytes) para suportar o programa correspondente a cada um dos quatro cenários, assumindo que a quantidade máxima de RAM endereçável é de 16K palavras.

SOLUÇÃO:

b) cenário “1-address-machine”

- formato de instrução: |opcode|endereço|
- assumindo que a ISA tem apenas 4 instruções (ADD, MULT, LOAD e STORE), são apenas necessários 2 bits para o opcode:
 $\#INST(ISA) = 4$ instruções
 $\#BITS(opcode) = \log_2(\#INST(ISA)) = \log_2(4) = 2$ bits
- todas as instruções necessitam de igual número de bits:
 - 2 bits para o opcode
 - 14 bits para 1 end. de memória (A a E, Temp)num total de 16 bits = 2 bytes
- o número de bits por instrução é fixo => todas as instruções terão 2 bytes
- contendo **7 instruções**, o programa ocupará $7 * 2 = 14$ bytes

Load B
Add C
Store Temp
Load D
Add E
Mult Temp
Store A
1-address machine

NOIS2 Recorde o exercício NOIS1 e apresente uma estimativa do espaço mínimo necessário (em bytes) para suportar o programa correspondente a cada um dos quatro cenários, assumindo que a quantidade máxima de RAM endereçável é de 16K palavras.

SOLUÇÃO:

c) cenário “2-address-machine”

- formatos de instrução: $\text{opcode|registro|endereço|}$ (*)
 $\text{opcode|registro|registro|}$ (**)
- assumindo que a ISA tem apenas 4 instruções (ADD, MULT, LOAD e STORE), são apenas necessários 2 bits para o opcode:
 $\#INST(ISA) = 4 \text{ instruções}$
 $\#BITS(opcode) = \log_2(\#INST(ISA)) = \log_2(4) = 2 \text{ bits}$
- assumindo que apenas existem 2 registros (R1 e R2), é apenas necessário gastar 1 bit de cada vez que é preciso identificar 1 desses registros:
 $\#REGISTOS(ISA) = 2 \text{ registros}$
 $\#BITS(id. \text{registro}) = \log_2(\#REGISTOS(ISA)) = \log_2(2) = 1 \text{ bit}$

Load R1, B	(*)
Add R1, C	(*)
Load R2, E	(*)
Add R2, E	(*)
Mult R2, R1	(**)
Store A, R2	(*)

2-address machine

NOIS2 Recorde o exercício NOIS1 e apresente uma estimativa do espaço mínimo necessário (em bytes) para suportar o programa correspondente a cada um dos quatro cenários, assumindo que a quantidade máxima de RAM endereçável é de 16K palavras.

SOLUÇÃO:

c) cenário “2-address-machine” (cont.)

- formatos de instrução: |opcode|registro|endereço| (*)
|opcode|registro|registro| (**)
 - as instruções que necessitam de mais bits são as instruções (*):
 - 2 bits para o opcode
 - 1 bit para o id. registro (R1 ou R2)
 - 14 bits para 1 endereço de memória (A a E)
num total de 17 bits, que arredondam para 24 (múltiplo de 8 mais próximo)
- o número de bits por instrução é fixo => todas as instruções terão 24 bits = 3 bytes
- contendo **6 instruções**, o programa ocupará $6 * 3 = 18$ bytes

Load R1, B	(*)
Add R1, C	(*)
Load R2, E	(*)
Add R2, E	(*)
Mult R2, R1	(**)
Store A, R2	(*)

2-address machine

NOIS2 Recorde o exercício NOIS1 e apresente uma estimativa do espaço mínimo necessário (em bytes) para suportar o programa correspondente a cada um dos quatro cenários, assumindo que a quantidade máxima de RAM endereçável é de 16K palavras.

SOLUÇÃO:

d) cenário “3-address-machine”

- formatos de instrução: $\text{opcode|registro|endereço|endereço|}$ (*)
 $\text{opcode|endereço|registro|registro|}$ (**)
- assumindo que a ISA tem apenas 2 instruções (ADD e MULT), é apenas necessário 1 bit para o opcode:
 $\#INST(ISA) = 2 \text{ instruções}$
 $\#BITS(opcode) = \log_2(\#INST(ISA)) = \log_2(2) = 1 \text{ bit}$
- assumindo que apenas existem 2 registros (R1 e R2), é apenas necessário gastar 1 bit de cada vez que é preciso identificar 1 desses registros:
 $\#REGISTOS(ISA) = 2 \text{ registros}$
 $\#BITS(id. \text{registro}) = \log_2(\#REGISTOS(ISA)) = \log_2(2) = 1 \text{ bit}$

Add R1, B, C	(*)
Add R2, D, E	(*)
Mult A, R1, R2	(**)
3-address machine	

NOIS2 Recorde o exercício NOIS1 e apresente uma estimativa do espaço mínimo necessário (em bytes) para suportar o programa correspondente a cada um dos quatro cenários, assumindo que a quantidade máxima de RAM endereçável é de 16K palavras.

SOLUÇÃO:

d) cenário “3-address-machine” (cont.)

- formatos de instrução: |opcode|registro|endereço|endereço| (*)
|opcode|endereço|registro|registro| (**)
- as instruções que necessitam de mais bits são as instruções (*):
 - 1 bit para o opcode
 - 1 bit para o id. registro (R1 ou R2)
 - 14+14=28 bits para 2 endereços de memória (A a E)
num total de 30 bits, que arredondam para 32 (múltiplo de 8 mais próximo)
- o número de bits por instrução é fixo => todas as instruções terão 32 bits = 4 bytes
- contendo **3 instruções**, o programa ocupará $3 * 4 = 12$ bytes

Add R1, B, C	(*)
Add R2, D, E	(*)
Mult A, R1, R2	(**)
3-address machine	

NOIS2 Recorde o exercício NOIS1 e apresente uma estimativa do espaço mínimo necessário (em bytes) para suportar o programa correspondente a cada um dos quatro cenários, assumindo que a quantidade máxima de RAM endereçável é de 16K palavras.

Add R1, B, C
Add R2, D, E
Mult A, R1, R2

3-address machine

Load R1, B
Add R1, C
Load R2, E
Add R2, E
Mult R2, R1
Store A, R2

2-address machine

Load B
Add C
Store Temp
Load D
Add E
Mult Temp
Store A

1-address machine

Push B
Push C
Add
Push D
Push E
Add
Mult
Store A

0-address machine

SOLUÇÃO
(resumo):

cenário	0-addr	1-addr	2-addr	3-addr
#instruções	8	7	6	3 (3/8=37,5%)
#bytes	16	14	18	12 (12/16=75%)

- menos instruções não implicam necessariamente menos espaço
(menos instruções necessárias => cada instrução é mais longa)
- os rácios de instruções e de espaço não são necessariamente iguais

NOIS3 Considere o código abaixo, apropriado à execução numa arquitetura baseada em stack. Assuma que este código é executado num computador com uma memória 128 x 16 bits, com um formato de instrução do tipo |opcode| endereço|, e que qualquer instrução ocupará uma palavra de memória.

Apresente as quantidades solicitadas abaixo, sob a forma de um único número inteiro (não apresente o resultado sob a forma de produto, ou usando potências, ou outras formas abreviadas).

```
PUSH X
PUSH Y
ADD
PUSH W
PUSH Z
DIV
MUL
PUSH V
PUSH U
SUBT
DIV
NOT
POP Q
```

- a)** Indique o número total de bits (comprimento) das instruções.
- b)** Indique o número de bits do campo "endereço" das instruções, de forma a garantir a possibilidade de aceder a qualquer endereço da memória do computador.
- c)** Levando em conta os valores determinados em a) e b), indique o número de bits do campo "opcode" das instruções.
- d)** Indique o número de opcodes (diferentes) possíveis suportados pelo formato de instrução em causa.
- e)** Indique o número de opcodes (diferentes) efetivamente necessários para suportar o programa fornecido.
- f)** Indique o número de bytes ocupados pelo programa fornecido (incluindo as variáveis usadas pelo mesmo).

NOIS3 Considere o código abaixo, apropriado à execução numa arquitetura baseada em stack. Assuma que este código é executado num computador com uma memória 128 x 16 bits, com um formato de instrução do tipo |opcode| endereço|, e que qualquer instrução ocupará uma palavra de memória.

```
PUSH V  
PUSH U  
SUBT  
PUSH W  
PUSH Z  
DIV  
PUSH X  
PUSH Y  
ADD  
MUL  
DIV  
NOT  
POP Q
```

a) indique o número total de bits (comprimento) das instruções.

Solução: 16 bits

Justificação: se “qualquer instrução ocupará uma palavra de memória” e tendo este computador uma memória 128 x 16 bits, é razoável assumir que a dimensão da palavra será de 16 bits e portanto será também esse o número de bits das instruções.

NOIS3 Considere o código abaixo, apropriado à execução numa arquitetura baseada em stack. Assuma que este código é executado num computador com uma memória 128 x 16 bits, com um formato de instrução do tipo |opcode| endereço|, e que qualquer instrução ocupará uma palavra de memória.

```
PUSH V
PUSH U
SUBT
PUSH W
PUSH Z
DIV
PUSH X
PUSH Y
ADD
MUL
DIV
NOT
POP Q
```

b) Indique o número de bits do campo "endereço" das instruções, de forma a garantir a possibilidade de aceder a qualquer endereço da memória do computador.

Solução: 7 bits

Justificação: tendo este computador uma memória 128 x 16 bits, e assumindo endereçamento à palavra de 16 bits, então a memória terá 128 palavras, de endereços 0 a 127; esses endereços, em binário, escrevem-se com $\log_2(128) = 7$ bits.

NOIS3 Considere o código abaixo, apropriado à execução numa arquitetura baseada em stack. Assuma que este código é executado num computador com uma memória 128 x 16 bits, com um formato de instrução do tipo |opcode|endereço|, e que qualquer instrução ocupará uma palavra de memória.

```
PUSH V
PUSH U
SUBT
PUSH W
PUSH Z
DIV
PUSH X
PUSH Y
ADD
MUL
DIV
NOT
POP Q
```

c) Levando em conta os valores determinados em a) e b), indique o número de bits do campo "opcode" das instruções.

Solução: 9 bits

Justificação: este computador tem um formato de instrução do tipo |opcode|endereço| ; já sabemos, pela alínea a), que uma instrução ocupa uma palavra de memória, ou seja 16 bits; e também sabemos, pela alínea b), que numa instrução, o campo |endereço| ocupa 7 bits; donde, o campo |opcode| ocupará $16 - 7 = 9$ bits.

NOIS3 Considere o código abaixo, apropriado à execução numa arquitetura baseada em stack. Assuma que este código é executado num computador com uma memória 128 x 16 bits, com um formato de instrução do tipo |opcode| endereço|, e que qualquer instrução ocupará uma palavra de memória.

```
PUSH V
PUSH U
SUBT
PUSH W
PUSH Z
DIV
PUSH X
PUSH Y
ADD
MUL
DIV
NOT
POP Q
```

d) Indique o número de opcodes (diferentes) possíveis suportados pelo formato de instrução em causa.

Solução: **512** opcodes

Justificação: com base na alínea c), sabemos que o campo |opcode| tem 9 bits; donde, seria possível acomodar nesse campo $2^9 = 512$ combinações diferentes de 9 bits, ou seja, 512 opcodes diferentes (sendo verdade que neste programa apenas usamos 7 instruções diferentes, ou seja, usamos apenas 7 das 512 que seriam possíveis).

NOIS3 Considere o código abaixo, apropriado à execução numa arquitetura baseada em stack. Assuma que este código é executado num computador com uma memória 128 x 16 bits, com um formato de instrução do tipo |opcode| endereço|, e que qualquer instrução ocupará uma palavra de memória.

```
PUSH V
PUSH U
SUBT
PUSH W
PUSH Z
DIV
PUSH X
PUSH Y
ADD
MUL
DIV
NOT
POP Q
```

e) Indique o número de opcodes (diferentes) efetivamente necessários para suportar o programa fornecido.

Solução: **7** opcodes

Justificação: o programa fornecido tem 7 instruções diferentes (PUSH, SUBT, DIV, ADD, MUL, NOT e POP), pelo que se apenas escrevêssemos programas com essas 7 instruções, apenas precisaríamos de 7 opcodes diferentes (um por instrução diferente).

NOIS3 Considere o código abaixo, apropriado à execução numa arquitetura baseada em stack. Assuma que este código é executado num computador com uma memória 128 x 16 bits, com um formato de instrução do tipo |opcode| endereço|, e que qualquer instrução ocupará uma palavra de memória.

```
PUSH V  
PUSH U  
SUBT  
PUSH W  
PUSH Z  
DIV  
PUSH X  
PUSH Y  
ADD  
MUL  
DIV  
NOT  
POP Q
```

f) Indique o número de bytes ocupados pelo programa fornecido (incluindo as variáveis usadas pelo mesmo).

Solução: **26+14=40** bytes

Justificação: com base no enunciado sabemos que “qualquer instrução ocupará uma palavra de memória” e já assumimos que cada palavra corresponde a 16 bits, ou seja, 2 bytes; como o programa tem ao todo 13 instruções, então estas necessitam de $13 * 2 = 26$ bytes; adicionalmente, o programa usa 7 variáveis diferentes, sendo razoável assumir que cada variável ocupa também uma palavra de memória, donde as 7 variáveis usam $7 * 2 = 14$ bytes.

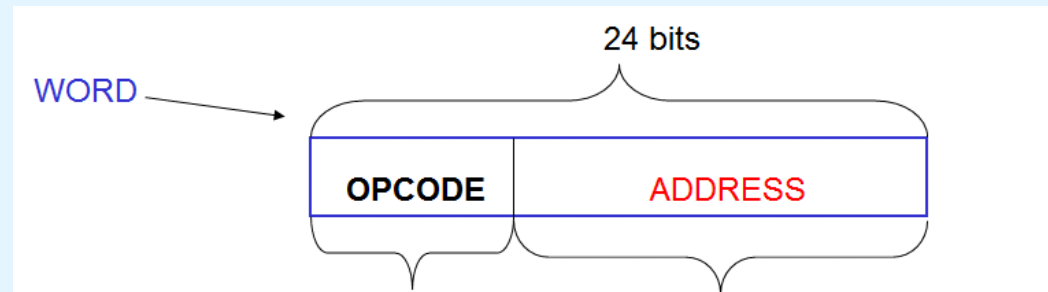
NOIS4 O conjunto de instruções de um certo computador tem 150 instruções diferentes, sendo que cada instrução inclui um opcode e um endereço de memória, e ocupa uma palavra de 24 bits.

- a) Quantos bits são necessários para o opcode ?**
- b) Quantos bits sobram para o endereço ?**
- c) Qual o tamanho máximo (em bytes) da memória ?**
- d) Qual o maior número inteiro (sem sinal) representável?**

NOIS4 O conjunto de instruções de um certo computador tem 150 instruções diferentes, sendo que cada instrução inclui um opcode e um endereço de memória, e ocupa uma palavra de 24 bits.

- a) Quantos bits são necessários para o opcode ?
- b) Quantos bits sobram para o endereço ?
- c) Qual o tamanho máximo (em bytes) da memória ?
- d) Qual o maior número inteiro (sem sinal) representável?

Dados do Problema:



$\#INSTRUÇÕES(ISA) = 150$ instruções

instrução = < **opcode**, **endereço** >

$\#BITS(instrução) = \#BITS(\text{palavra}) = 24$ bits

NOIS4 O conjunto de instruções de um certo computador tem 150 instruções diferentes, sendo que cada instrução inclui um opcode e um endereço de memória, e ocupa uma palavra de 24 bits.

a) Quantos bits são necessários para o opcode ?

Solução:

$$\begin{aligned}\text{\#BITS}(\text{opcode}) &\geq \log_2 (\text{\#INSTRUÇÕES(ISA)}) \\ &= \log_2(150) = 7, \dots\end{aligned}$$

donde,

$$\text{\#BITS}_{\min}(\text{opcode}) = \text{ceil}(7, \dots) = \mathbf{8 \text{ bits}}$$

NOIS4 O conjunto de instruções de um certo computador tem 150 instruções diferentes, sendo que cada instrução inclui um opcode e um endereço de memória, e ocupa uma palavra de 24 bits.

b) Quantos bits sobram para o endereço ?

Solução:

$$\#BITS(\text{instrução}) = \#BITS(\text{opcode}) + \#BITS(\text{endereço})$$

donde,

$$\begin{aligned}\#BITS(\text{endereço}) &= \#BITS(\text{instrução}) - \#BITS(\text{opcode}) \\ &= 24 - 8 \\ &= \mathbf{16 \text{ bits}}\end{aligned}$$

NOIS4 O conjunto de instruções de um certo computador tem 150 instruções diferentes, sendo que cada instrução inclui um opcode e um endereço de memória, e ocupa uma palavra de 24 bits.

c) Qual o tamanho máximo (em bytes) da memória ?

Solução:

$$\begin{aligned}\#PALAVRAS_{\max}(\text{RAM}) &= 2^{\#BITS(\text{endereço})} \\ &= 2^{16} \\ &= 64\text{K palavras}\end{aligned}$$

$$\#BYTES(\text{palavra}) = \#BITS(\text{palavra}) / 8 \text{ bits} = 24 / 8 = 3 \text{ bytes}$$

$$\begin{aligned}\#BYTES_{\max}(\text{RAM}) &= \#PALAVRAS_{\max}(\text{RAM}) \times \#BYTES(\text{palavra}) \\ &= 64\text{K} \times 3 \\ &= \mathbf{192 \text{ Kbytes}}\end{aligned}$$

NOIS4 O conjunto de instruções de um certo computador tem 150 instruções diferentes, sendo que cada instrução inclui um opcode e um endereço de memória, e ocupa uma palavra de 24 bits.

d) Qual o maior número inteiro (sem sinal) representável ?

Solução:

#BITS(palavra)=24 bits

donde,

o maior inteiro (sem sinal) representável será **$2^{24} - 1$**

NOIS5 Um computador tem uma memória de 256K palavras de 32 bits cada. Cada instrução desse computador ocupa uma palavra e tem 4 componentes: opcode, modo de endereçamento (1 de 7 possíveis), endereço de registo (1 de 60 possíveis) e endereço de memória.

- a) Quantos bits são necessários para o modo de end. ?**
- b) Quantos bits são necessários para o end. de registo?**
- c) Quantos bits são necessários para o end. de memória (assumindo que o endereçamento é feito à palavra) ?**
- d) Quantos bits sobram para o opcode? E quantas instruções diferentes são possíveis ?**

NOIS5 Um computador tem uma memória de 256K palavras de 32 bits cada. Cada instrução desse computador ocupa uma palavra e tem 4 componentes: opcode, modo de endereçamento (1 de 7 possíveis), endereço de registo (1 de 60 possíveis) e endereço de memória.

- a) Quantos bits são necessários para o modo de end. ?
- b) Quantos bits são necessários para o end. de registo?
- c) Quantos bits são necessários para o end. de memória (assumindo que o endereçamento é feito à palavra) ?
- d) Quantos bits sobram para o opcode? E quantas instruções diferentes são possíveis ?

Dados do Problema:

#PALAVRAS(RAM)=**256K** palavras

#BITS(inst) = #BITS(palavra RAM)= 32 bits

instrução=<opcode,
modo end. (1-**7**),
end. registo (1-**60**),
end. RAM>

NOIS5 Um computador tem uma memória de 256K palavras de 32 bits cada. Cada instrução desse computador ocupa uma palavra e tem 4 componentes: opcode, modo de endereçamento (1 de 7 possíveis), endereço de registo (1 de 60 possíveis) e endereço de memória.

a) Quantos bits são necessários para o modo de end. ?

Solução:

$$\#BITS(\text{modo end.}) \geq \log_2(7) = 2,...$$

donde,

$$\#BITS_{\min}(\text{modo end.}) = \text{ceil}(2,...) = \mathbf{3 \text{ bits}}$$

NOIS5 Um computador tem uma memória de 256K palavras de 32 bits cada. Cada instrução desse computador ocupa uma palavra e tem 4 componentes: opcode, modo de endereçamento (1 de 7 possíveis), endereço de registo (1 de 60 possíveis) e endereço de memória.

b) Quantos bits são necessários para o end. de registo ?

Solução:

$$\#BITS(\text{end. registo}) \geq \log_2(60) = 5,...$$

donde,

$$\#BITS_{\min}(\text{end. registo}) = \text{ceil}(5, ...) = \mathbf{6 \text{ bits}}$$

NOIS5 Um computador tem uma memória de 256K palavras de 32 bits cada. Cada instrução desse computador ocupa uma palavra e tem 4 componentes: opcode, modo de endereçamento (1 de 7 possíveis), endereço de registo (1 de 60 possíveis) e endereço de memória.

c) Quantos bits são necessários para o end. de memória (assumindo que o endereçamento é feito à palavra) ?

Solução:

$$\begin{aligned}\#BITS(\text{end. RAM}) &\geq \log_2(256K) = \log_2(2^8 \times 2^{10}) \\ &= 18 \text{ bits}\end{aligned}$$

donde,

$$\#BITS_{\min}(\text{end. RAM}) = \text{ceil}(18) = \mathbf{18 \text{ bits}}$$

NOIS5 Um computador tem uma memória de 256K palavras de 32 bits cada. Cada instrução desse computador ocupa uma palavra e tem 4 componentes: opcode, modo de endereçamento (1 de 7 possíveis), endereço de registo (1 de 60 possíveis) e endereço de memória.

d) Quantos bits sobram para o opcode ?

E quantas instruções diferentes são possíveis ?

Solução:

$$\begin{aligned}\#BITS(inst) &= \#BITS(opcode) + \#BITS(modo\ end.) + \\ &\quad \#BITS(end.\ registo) + \#BITS(end.\ RAM) \\ &= \#BITS(opcode) + 3 + 6 + 18 = 32\ bits\end{aligned}$$

donde,

$$\#BITS_{(opcode)} = 32 - (3 + 6 + 18) = 32 - 27 = \mathbf{5\ bits}$$

(e, em consequência, $\#INST_{max}(ISA) = 2^5 = 32$ instruções)

EO1 Um computador tem instruções de 32 bits, endereços de memória de 12 bits e já suporta 250 instruções com 2 endereços.
a) Quantas instruções com 1 endereço são possíveis de definir ?

Nota: apresente também o algoritmo de decodificação correspondente.

EO1 Um computador tem instruções de 32 bits, endereços de memória de 12 bits e já suporta 250 instruções com 2 endereços.
a) Quantas instruções com 1 endereço são possíveis de definir ?

Solução:

00000000	xxxxxxxxxxxx	xxxxxxxxxxxx	}	250 2-address instructions
11111001	xxxxxxxxxxxx	xxxxxxxxxxxx		
11111010	000000000000	xxxxxxxxxxxx	}	2^{12}
11111010	111111111111	xxxxxxxxxxxx		
...			}	$6 * 2^{12}$ 1-address instructions
11111111	000000000000	xxxxxxxxxxxx		
11111111	111111111111	xxxxxxxxxxxx		

Algoritmo de decodificação:

```
if (leftmost 8 bits < 11111010) {  
    execute 2-address instruction  
}  
else {  
    execute 1-address instruction  
}
```

EO1 Um computador tem instruções de 32 bits, endereços de memória de 12 bits e já suporta 250 instruções com 2 endereços.
b) Quantas instruções com 0 endereços são possíveis de definir ?

Nota: apresente também o algoritmo de decodificação correspondente.

EO1 Um computador tem instruções de 32 bits, endereços de memória de 12 bits e já suporta 250 instruções com 2 endereços.
b) Quantas instruções com 0 endereços são possíveis de definir

Solução:

Considerando a solução da alínea a), se os últimos 12 bits ficarem também disponíveis para o *opcode*, então a seguir aos primeiros 8 bits (que só já podem variar 6 vezes), há 24 bits que podem variar completamente (2^{24} vezes), donde o número total de instruções possíveis com 0 endereços é de:

6×2^{24} instruções

Algoritmo de decodificação:

```
if (leftmost 8 bits < 11111010) {  
    execute 2-address instruction  
}  
else {  
    execute 0-address instruction  
}
```

EO2 Um computador tem instruções de 11 bits e endereços de 4 bits. a) Verifique se é possível suportar 5 instruções de 2 endereços, 45 instruções de 1 endereço e ainda 32 instruções de 0 endereços.

Nota: apresente também o algoritmo de decodificação correspondente.

EO2 Um computador tem instruções de 11 bits e endereços de 4 bits. a) Verifique se é possível suportar 5 instruções de 2 endereços, 45 instruções de 1 endereço e ainda 32 instruções de 0 endereços.

Solução:

000	xxxx	xxxx	}	5 2-address
100	xxxx	xxxx		
101	<u>0000</u>	xxxx	}	16
101	<u>1111</u>	xxxx		
110	<u>0000</u>	xxxx	}	16
110	<u>1111</u>	xxxx		
111	<u>0000</u>	xxxx	}	13
111	<u>1100</u>	xxxx		
111	<u>1110</u>	0000	}	16
111	<u>1101</u>	1111		
111	<u>1110</u>	0000	}	16
111	<u>1111</u>	1111		

45 1-address

32 0-address

Algoritmo de decodificação:

```
if (leftmost 3 bits < 101) {  
    execute 2-address instruction  
}  
else if (leftmost 6 bits < 111111) {  
    execute 1-address instruction  
}  
else {  
    execute 0-address instruction  
}
```

EO2 Um computador tem instruções de 11 bits e endereços de 4 bits. b) Assuma que já foram definidas 6 instruções de 2 endereços e 24 instruções de 0 endereços; nestas condições de partida, quantas instruções de 1 endereço são ainda possíveis de definir?

Nota: apresente também o algoritmo de decodificação correspondente.

EO2 Um computador tem instruções de 11 bits e endereços de 4 bits. b) Assuma que já foram definidas 6 instruções de 2 endereços e 24 instruções de 0 endereços; nestas condições de partida, quantas instruções de 1 endereço são ainda possíveis de definir?

Solução:

Algoritmo de decodificação:

```

if (leftmost 3 bits < 110) {
    execute 2-address instruction
}
else if (leftmost 6 bits < 111111) {
    execute 1-address instruction
}
else {
    execute 0-address instruction
}
    
```

<u>000</u>	xxxx	xxxx	}	6 2-addresses
...				
<u>101</u>	xxxx	xxxx		

110	<u>0000</u>	xxxx	}	16 1-addresses
	...			
110	<u>1111</u>	xxxx		

111	<u>0000</u>	xxxx	}	14 1-addresses
	...			
111	<u>1101</u>	xxxx		

111	1110	<u>1000</u>	}	24 0-addresses
		...		
111	1110	<u>1111</u>		
111	1111	<u>0000</u>		
		...		
111	1111	<u>1111</u>		

EO3 Um computador tem instruções de 24 bits e endereços (de RAM) de 8 bits. Com 34 instruções de 2 endereços já definidas, quantas instruções seria possível ainda acomodar com espaço para a) 1 endereço, b) 0 endereços ? (considere a) e b) independentes)

EO3 Um computador tem instruções de 24 bits e endereços (de RAM) de 8 bits. Com 34 instruções de 2 endereços já definidas, quantas instruções seria possível ainda acomodar com espaço para a) 1 endereço, b) 0 endereços ? (considere a) e b) independentes)

Solução:

Com instruções de 24 bits e endereços de 8 bits, uma instrução de 2 endereços tem o formato | opcode (8 bits) | endereço1 (8 bits) | endereço2 (8 bits) | , em que $8 = 24 - (8 + 8)$.

Havendo já 34 instruções de 2 endereços, e assumindo um consumo sequencial das combinações de bits disponíveis para o opcode, as combinações 0 a 33 foram usadas para instruções de 2 endereços, ou seja:

opcode (8 bits)	endereço1 (8 bits)	endereço2 (8 bits)
0 = 00000000	XXXXXXXX	XXXXXXXX
...		
33 = 00100001	XXXXXXXX	XXXXXXXX

EO3 Um computador tem instruções de 24 bits e endereços (de RAM) de 8 bits. Com 34 instruções de 2 endereços já definidas, quantas instruções seria possível ainda acomodar com espaço para a) 1 endereço, b) 0 endereços ? (considere a) e b) independentes)

Solução:

Com instruções de 24 bits e endereços de 8 bits, uma instrução de 2 endereços tem o formato | opcode (8 bits) | endereço1 (8 bits) | endereço2 (8 bits) | , em que $8 = 24 - (8 + 8)$.

Havendo já 34 instruções de 2 endereços, e assumindo um consumo sequencial das combinações de bits disponíveis para o opcode, as combinações 0 a 33 foram usadas para instruções de 2 endereços, ou seja:

opcode (8 bits)	endereço1 (8 bits)	endereço2 (8 bits)
0 = 00000000	XXXXXXXX	XXXXXXXX
...		
33 = 00100001	XXXXXXXX	XXXXXXXX

a) Uma instrução de 1 endereço terá o formato | opcode (8+8 bits) | endereço (8 bits) |. Porém, os primeiros 8 bits (prefixo) do opcode só poderão variar de 34 a 255, ao passo que os segundos 8 bits (sufixo) podem variar completamente (de 0 a $2^8-1=255$). Assim, por cada prefixo entre 34 e 255, o opcode pode ter $2^8=256$ sufixos (de 0 a 255), o que dá um total de $(255-34+1) * 256 = 56832$ instruções de 1 endereço.

prefixo do opcode (8 bits)	sufixo do opcode (8 bits)	endereço (8 bits)
34 = 00100010	2^8 combinações: 0 a 255	XXXXXXXX
...		
255 = 11111111	2^8 combinações: 0 a 255	XXXXXXXX

EO3 Um computador tem instruções de 24 bits e endereços (de RAM) de 8 bits. Com 34 instruções de 2 endereços já definidas, quantas instruções seria possível ainda acomodar com espaço para a) 1 endereço, b) 0 endereços ? (considere a) e b) independentes)

Solução:

Com instruções de 24 bits e endereços de 8 bits, uma instrução de 2 endereços tem o formato | opcode (8 bits) | endereço1 (8 bits) | endereço 2 (8 bits) | , em que $8 = 24 - (8 + 8)$.

Havendo já 34 instruções de 2 endereços, e assumindo um consumo sequencial das combinações de bits disponíveis para o opcode, as combinações 0 a 33 foram usadas para instruções de 2 endereços, ou seja:

opcode (8 bits)	endereço1 (8 bits)	endereço2 (8 bits)
0 = 00000000	XXXXXXXX	XXXXXXXX
...		
33 = 00100001	XXXXXXXX	XXXXXXXX

b) Uma instrução de 0 endereços terá o formato | opcode (8+16 bits) | , em que os 8 bits do prefixo do opcode só já poderão variar de 34 a 255, ao passo que os 16 bits do sufixo podem variar completamente (de 0 a $2^{16}-1=65535$). Assim, por cada prefixo entre 34 e 255, o opcode pode ter $2^{16}=65536$ sufixos (de 0 a 65536), o que dá um total de $(255-34+1) * 65536 = 14548992$ instruções de 0 endereços.

prefixo do opcode (8 bits)	sufixo do opcode (16 bits)	
34 = 00100010	2^{16} combinações: 0 a 65535	
...		
255 = 11111111	2^{16} combinações: 0 a 65535	

EO4 Um computador tem instruções de 32 bits e endereços (de RAM) de 8 bits. Responda às seguintes questões:

- a)** Mostre como consumiria as combinações disponíveis de 32 bits para acomodar, em simultâneo, 64 instruções com 3 endereços e 256 instruções de 2 endereços.
- b)** Usando como ponto de partida o resultado da alínea a), quantas instruções com 1 endereço seriam possíveis de acomodar ? (justifique mostrando como se consomem as combinações de 32 bits que sobraram da alínea a)).
- c)** Usando como ponto de partida o resultado da alínea a), quantas instruções com 0 endereços seriam possíveis de acomodar ? (justifique mostrando como se consomem as combinações de 32 bits que sobraram da alínea a)).

NOTA:

consoma as combinações de bits por ordem crescente (de 32 0's para 32 1's).

EO4 Um computador tem instruções de 32 bits e endereços (de RAM) de 8 bits. Responda às seguintes questões:

a) Mostre como consumiria as combinações disponíveis de 32 bits para acomodar, em simultâneo, 64 instruções com 3 endereços e 256 instruções de 2 endereços.

Solução:

Com instruções de 32 bits e endereços de 8 bits, uma instrução de 3 endereços tem o formato | opcode (8 bits) | endereço1 (8 bits) | endereço2 (8 bits) | endereço3 (8 bits) |, em que $8 = 32 - (8 + 8 + 8)$. Assumindo um consumo sequencial das combinações de bits disponíveis para o opcode, as combinações 0 a 63 asseguram 64 instruções de 3 endereços, ou seja:

opcode (8 bits)	endereço1 (8 bits)	endereço2 (8 bits)	endereço3 (8 bits)
0 = 00000000	XXXXXXXX	XXXXXXXX	XXXXXXXX
...			
63 = 00111111	XXXXXXXX	XXXXXXXX	XXXXXXXX

Uma instrução de 2 endereços terá o formato | opcode (8+8 bits) | endereço1 (8 bits) | endereço2 (8 bits) |. Continuando a consumir as combinações de bits ainda disponíveis, os 8 bits do prefixo do opcode podem ainda variar de 64 a 255, e os 8 bits do sufixo podem variar completamente (de 0 a $2^8-1=255$). Assim, é possível acomodar $(255-64+1) * 2^8 = 49152$ instruções de 2 endereços, bem mais que as 256 pedidas !

prefixo do opcode (8 bits)	sufixo do opcode (8 bits)	endereço1 (8 bits)	endereço2 (8 bits)
64 = 01000000	2^8 combinações: 0 a 255	XXXXXXXX	XXXXXXXX
...			
255 = 11111111	2^8 combinações: 0 a 255	XXXXXXXX	XXXXXXXX

EO4 Um computador tem instruções de 32 bits e endereços (de RAM) de 8 bits. Responda às seguintes questões:

b) Usando como ponto de partida o resultado da alínea a), quantas instruções com 1 endereço seriam possíveis de acomodar ? (justifique mostrando como se consomem as combinações de 32 bits que sobraram da alínea a)).

Solução:

Ponto de partida (64 instruções de 3 endereços, e 256 instruções de 2 endereços):

opcode (8 bits)	endereço1 (8 bits)	endereço2 (8 bits)	endereço3 (8 bits)
0 = 00000000	XXXXXXXX	XXXXXXXX	XXXXXXXX
...			
63 = 00111111	XXXXXXXX	XXXXXXXX	XXXXXXXX

prefixo do opcode (8 bits)	sufixo do opcode (8 bits)	endereço1 (8 bits)	endereço2 (8 bits)
64 = 01000000	<u>2⁸</u> combinações: 0 a 255	XXXXXXXX	XXXXXXXX

Uma instrução de 1 endereço terá o formato | opcode (8+16 bits) | endereço (8 bits) |. O prefixo de 8 bits do opcode poderá variar de 65 a 255, e o sufixo de 16 bits poderá variar completamente (0 a 2¹⁶-1=65535):

prefixo do opcode (8 bits)	sufixo do opcode (16 bits)	endereço (8 bits)
65 = 01000001	<u>2¹⁶</u> combinações: 0 a 65535	XXXXXXXX
...		
255 = 11111111	<u>2¹⁶</u> combinações: 0 a 65535	XXXXXXXX

Será ainda possível acomodar $(255-65+1)*2^{16} = 12517376$ instruções de 1 endereço.

EO4 Um computador tem instruções de 32 bits e endereços (de RAM) de 8 bits. Responda às seguintes questões:

c) Usando como ponto de partida o resultado da alínea a), quantas instruções com 0 endereços seriam possíveis de acomodar ? (justifique mostrando como se consomem as combinações de 32 bits que sobraram da alínea a)).

Solução:

Ponto de partida (64 instruções de 3 endereços, e 256 instruções de 2 endereços):

opcode (8 bits)	endereço1 (8 bits)	endereço2 (8 bits)	endereço3 (8 bits)
0 = 00000000	XXXXXXXX	XXXXXXXX	XXXXXXXX
...			
63 = 00111111	XXXXXXXX	XXXXXXXX	XXXXXXXX
prefixo do opcode (8 bits)	sufixo do opcode (8 bits)	endereço1 (8 bits)	endereço2 (8 bits)
64 = 01000000	2^8 combinações: 0 a 255	XXXXXXXX	XXXXXXXX

Uma instrução de 0 endereços terá o formato | opcode ($8+24$ bits) |. O prefixo de 8 bits do opcode poderá variar de 65 a 255, e o sufixo de 24 bits poderá variar completamente (0 a $2^{24}-1=16777215$):

prefixo do opcode (8 bits)	sufixo do opcode (24 bits)	
65 = 01000001	2^{24} combinações: 0 a 16777215	
...		
255 = 11111111	2^{24} combinações: 0 a 16777215	

Será ainda possível acomodar $(255-65+1)*2^{24} = 3204448256$ instruções de 0 endereços.

AM1 Considere a instrução LOAD 1000. Para a memória e para o registo R1 com o conteúdo indicado, e assumindo que R1 é usado implicitamente no modo de endereçamento indexado, complete a tabela com os valores (em falta) a carregar no registo AC.

Memory

1000	1400
...	
1100	400
...	
1200	1000
...	
1300	1100
...	
1400	1300

R1

200

Mode	Value Loaded into AC
Immediate	
Direct	
Indirect	
Indexed	

AM1 Considere a instrução LOAD 1000. Para a memória e para o registo R1 com o conteúdo indicado, e assumindo que R1 é usado implicitamente no modo de endereçamento indexado, complete a tabela com os valores (em falta) a carregar no registo AC.

Memory

1000	1400
...	
1100	400
...	
1200	1000
...	
1300	1100
...	
1400	1300

R1

200

Solução:

Mode	Value
Immediate	1000
Direct	1400
Indirect	1300
Indexed	1000

AM2 Considere a instrução LOAD 500. Para a memória e para o registo R1 com o conteúdo indicado, e assumindo que R1 é usado implicitamente no modo de endereçamento indexado, complete a tabela com os valores (em falta) a carregar no registo AC.

100	600	R1 200
...		
400	300	
...		
500	100	
...		
600	500	
...		
700	800	

Mode	Value Loaded into AC
Immediate	
Direct	
Indirect	
Indexed	

AM2 Considere a instrução LOAD 500. Para a memória e para o registo R1 com o conteúdo indicado, e assumindo que R1 é usado implicitamente no modo de endereçamento indexado, complete a tabela com os valores (em falta) a carregar no registo AC.

100	600	R1 200
...		
400	300	
...		
500	100	
...		
600	500	
...		
700	800	

Solução:

Mode	Value
Immediate	500
Direct	100
Indirect	600
Indexed	800

AM3 Uma CPU de uma arquitetura baseada em registo Acumulador (AC) suporta múltiplos modos de endereçamento e o conteúdo da memória RAM num certo instante é o representado pela figura abaixo.

Para a execução de uma instrução LOAD 1200, e assumindo a existência de um registo R1200 com conteúdo 1800 usado nos modos de endereçamento que necessitem de um registo auxiliar, indique o valor numérico que será transferido para o registo AC (acumulador) se o modo de endereçamento for:

- a) Imediato
- b) Direto
- c) Direto por Registo (usando o registo R1200)
- d) Indireto
- e) Indireto por Registo (usando o registo R1200)
- f) Indexado (usando o registo R1200)
- g) com Base (usando o registo R1200)

endereço	conteúdo
1200	1400
1400	1600
1600	1800
1800	2000
2000	1200
3000	1200

AM3 Uma CPU de uma arquitetura baseada em registo Acumulador (AC) suporta múltiplos modos de endereçamento e o conteúdo da memória RAM num certo instante é o representado pela figura abaixo.

Solução:

Para a execução de uma instrução LOAD 1200, e assumindo a existência de um registo R1200 com conteúdo 1800 usado nos modos de endereçamento que necessitem de um registo auxiliar, indique o valor numérico que será transferido para o registo AC (acumulador) se o modo de endereçamento for:

a) Imediato: AC = 1200

b) Direto: AC = M[1200] = 1400

c) Direto por Registo (usando o R1200): AC = R1200 = 1800

d) Indireto: AC = M[M[1200]] = M[1400] = 1600

e) Indireto por Registo (usando o R1200): AC = M[R1200] = M[1800] = 2000

f) Indexado (usando o R1200): AC = M [Base + Deslocamento]
= M [1200 + R1200] = M [1200 + 1800]
= M[3000] = 1200

g) com Base (usando o R1200): AC = M [Base + Deslocamento]
= M [R1200 + 1200] = M [1800 + 1200]
= M[3000] = 1200

endereço	conteúdo
1200	1400
1400	1600
1600	1800
1800	2000
2000	1200
3000	1200

ILP1 A CPU de um sistema possui uma pipeline de 5 estágios, com ciclo de relógio de 40ns. Sem recorrer à pipeline, o processamento de uma instrução nessa CPU demora, em média, 200ns. Determine (*): a) o *speedup* oferecido pela pipeline para a execução de um programa com 400 instruções; b) o máximo *speedup* teórico da pipeline.

$$S_n = (n \times t_i) / [(k + n - 1) \times t_s] \\ = (n \times k) / (k + n - 1)$$

$$\text{MAX}(S_n) = k$$

com

n = nº de instruções

t_i = tempo da instrução

k = nº de estágios da pipeline

t_s = tempo de cada estágio

(*) Respostas truncadas às milésimas

ILP1 A CPU de um sistema possui uma pipeline de 5 estágios, com ciclo de relógio de 40ns. Sem recorrer à pipeline, o processamento de uma instrução nessa CPU demora, em média, 200ns. Determine (*): a) o *speedup* oferecido pela pipeline para a execução de um programa com 400 instruções; b) o máximo *speedup* teórico da pipeline.

Solução:

$n = 400$ instruções

$t_i = 200$ ns

$k = 5$ estágios

$t_s = 40$ ns

$$S_n = (n \times t_i) / [(k + n - 1) \times t_s] \\ = (n \times k) / (k + n - 1)$$

$$\text{MAX}(S_n) = k$$

com

$n = n^\circ$ de instruções

$t_i =$ tempo da instrução

$k = n^\circ$ de estágios da pipeline

$t_s =$ tempo de cada estágio

a) speedup da pipeline na execução de 400 instruções

$$S_{400} = (400 \times 5) / (5 + 400 - 1) = 4.950 (*)$$

(*) Respostas truncadas às milésimas

b) máximo speedup teórico da pipeline

$$\text{MAX}(S_n) = 5$$

ILP2 A CPU de um sistema possui uma pipeline de 5 estágios, com ciclo de relógio de 20ns. Sem recorrer à pipeline, o processamento de uma instrução nessa CPU demora, em média, 100ns. Determine (*): a) o *speedup* oferecido pela pipeline para a execução de um programa com 200 instruções; b) o máximo *speedup* teórico da pipeline.

$$S_n = (n \times t_i) / [(k + n - 1) \times t_s] \\ = (n \times k) / (k + n - 1)$$

$$\text{MAX}(S_n) = k$$

com

n = nº de instruções

t_i = tempo da instrução

k = nº de estágios da pipeline

t_s = tempo de cada estágio

(*) Respostas truncadas às milésimas

ILP2 A CPU de um sistema possui uma pipeline de 5 estágios, com ciclo de relógio de 20ns. Sem recorrer à pipeline, o processamento de uma instrução nessa CPU demora, em média, 100ns. Determine (*): a) o *speedup* oferecido pela pipeline para a execução de um programa com 200 instruções; b) o máximo *speedup* teórico da pipeline.

Solução:

$n = 200$ instruções

$t_i = 100$ ns

$k = 5$ estágios

$t_s = 20$ ns

$$S_n = (n \times t_i) / [(k + n - 1) \times t_s] \\ = (n \times k) / (k + n - 1)$$

$$\text{MAX}(S_n) = k$$

com

$n = n^\circ$ de instruções

$t_i =$ tempo da instrução

$k = n^\circ$ de estágios da pipeline

$t_s =$ tempo de cada estágio

a) speedup da pipeline na execução de 200 instruções

$$S_{200} = (200 \times 5) / (5 + 200 - 1) = 4.901 (*)$$

(*) Respostas truncadas às milésimas

b) máximo speedup teórico da pipeline

$$\text{MAX}(S_n) = 5$$

ILP3 Uma CPU demora 117 ns, em média, a processar uma instrução seguindo o modelo de von Neumann, em que cada instrução atravessa as fases de fetch, decode e execute, antes da próxima iniciar o mesmo percurso. Uma versão melhorada dessa CPU foi munida de uma pipeline de encadeamento de instruções, com 13 estágios e uma duração por estágio de 9 ns.

Apresente as quantidades solicitadas abaixo, sob a forma de um único número (não apresente o resultado como produto, com potências, ou outras formas abreviadas; apresente os valores (*) com precisão limitada às centésimas e truncados).

a) Tempo, em ns (nano-segundos), que demoraria a CPU original (não-melhorada) a executar um programa X com 4000 instruções.

b) Tempo, em ns, que demoraria a CPU melhorada a executar o programa X.

c) (*) Aceleração (*speedup*) proporcionada pela pipeline da CPU melhorada na execução do programa X.

d) (*) Eficiência com que a pipeline da CPU melhorada é usada durante a execução do programa X. Nota: a Eficiência é a percentagem que o valor obtido em c) representa em relação à aceleração máxima teórica proporcionada pela pipeline.

ILP3 Uma CPU demora 117 ns, em média, a processar uma instrução seguindo o modelo de von Neumann, em que cada instrução atravessa as fases de fetch, decode e execute, antes da próxima iniciar o mesmo percurso. Uma versão melhorada dessa CPU foi munida de uma pipeline de encadeamento de instruções, com 13 estágios e uma duração por estágio de 9 ns.

Apresente as quantidades solicitadas abaixo, sob a forma de um único número (não apresente o resultado como produto, com potências, ou outras formas abreviadas; apresente os valores (*) com precisão limitada às centésimas e truncados).

Solução:

a) Tempo, em ns (nano-segundos), que demoraria a CPU original (não-melhorada) a executar um programa X com 4000 instruções.

$$T_{\text{non-pipeline}} = n * t_i = 4000 * 117 = 468000 \text{ ns}$$

b) Tempo, em ns, que demoraria a CPU melhorada a executar o programa X.

$$T_{\text{pipeline}} = (k + n - 1) * t_s = (13 + 4000 - 1) * 9 = 36108 \text{ ns}$$

ILP3 Uma CPU demora 117 ns, em média, a processar uma instrução seguindo o modelo de von Neumann, em que cada instrução atravessa as fases de fetch, decode e execute, antes da próxima iniciar o mesmo percurso. Uma versão melhorada dessa CPU foi munida de uma pipeline de encadeamento de instruções, com 13 estágios e uma duração por estágio de 9 ns.

Apresente as quantidades solicitadas abaixo, sob a forma de um único número (não apresente o resultado como produto, com potências, ou outras formas abreviadas; apresente os valores (*) com precisão limitada às centésimas e truncados).

Solução:

c) (*) Aceleração (*speedup*) proporcionada pela pipeline da CPU melhorada na execução do programa X.

$$\text{Speedup}_X = T_{\text{non-pipelined}} / T_{\text{pipelined}} = 468000 / 36108 = 12,96$$

d) (*) Eficiência com que a pipeline da CPU melhorada é usada durante a execução do programa X. Nota: a Eficiência é a percentagem que o valor obtido em c) representa em relação à aceleração máxima teórica proporcionada pela pipeline.

$$\text{Efficiency}_X = \text{Speedup}_X / k = 12,96 / 13 = 99,69\%$$